

The Sea Shells of the Universe: Automating H I Shell Detection

ERIC CHEN ^{1,*} AND JOSEPH RODRIGUEZ ^{1,*}

¹*Department of Astronomy and Astrophysics, University of California, San Diego, La Jolla, CA 92093, USA*

ABSTRACT

Galaxies are full of holes. Supernovae and stellar winds push the neutral hydrogen (H I) into expanding shells that show up as rings and arcs in position-velocity (PV) slices. Cataloging these by eye is slow, so we built a pipeline, `hishells_pv`, that learns to flag them. We slice standardized PV cuts out of THINGS H I cubes, label them from the I. Bagetakos et al. (2011) hole catalog, train a high-recall U-Net to segment shell-like structure, calibrate a probability threshold, and project ranked candidates back onto the sky as per-galaxy maps. On held-out galaxies the network reaches a patch-level recall near 0.97 and an F1 near 0.88 at the calibrated threshold, the high-recall behavior we want from a first-pass finder. We also use the catalog to ask whether the Bagetakos Type 1/2/3 tracks real physics or is mostly a selection effect. Using Gaussian mixture clustering with BIC model selection, nested logistic regression, and a galaxy-stratified random forest, we find the labels line up with how well a hole is resolved much more than with any clean physical split, so the Type label is best treated as auxiliary metadata.

Keywords: Interstellar medium (847), H I line emission (690), Galaxy structure (622), Convolutional neural networks (1938), Astronomy data analysis (1858)

1. SCIENTIFIC MOTIVATION

The gas between the stars is not smooth. In star-forming galaxies, massive stars and their supernovae dump energy into the surrounding H I, sweeping the neutral gas into expanding bubbles that grow into shells and supershells hundreds of parsecs to a kiloparsec across (S. Ehlerová & J. Palouš 2005; R. Boomsma et al. 2008). These holes are fingerprints of stellar feedback, the same feedback that regulates star formation and keeps the interstellar medium turbulent (C. Bacchini et al. 2020). Their sizes and expansion speeds encode how much energy went into them and when, feeding the bigger question of how feedback shapes galaxies.

The cleanest way to spot an expanding shell is in a position-velocity (PV) cut: plot intensity against line-of-sight velocity along a thin slit, and an expanding shell appears as a ring or pair of arcs, since the near side moves toward you and the far side away. A stalled or blown-out hole shows a partial version. This is the basis of the I. Bagetakos et al. (2011) hole catalog that anchors this project: working from THINGS (F. Walter et al. 2008), they visually identified more than a thou-

sand H I holes across 20 nearby galaxies, measured diameters and expansion velocities (about 4 to 36 km s⁻¹ where measurable), and sorted each into one of three Types by kinematic signature.

We take on two problems. First, visual cataloging does not scale: THINGS has tens of galaxies, but upcoming surveys like WALLABY will have hundreds of thousands of sources (L. Wang et al. 2025), so finders must run on their own. After template matching and parametric shell searches (S. Mashchenko & N. St-Louis 2002; S. Ehlerová & J. Palouš 2005) and, more recently, convolutional neural networks (C. M. Van Oort et al. 2019; L. Wang et al. 2025), we build a high-recall segmentation model that flags shell-like structure in PV space so a human only vets a short ranked list.

Second, we stress-test the catalog itself. Following E. Brinks & E. Bajaja (1986), I. Bagetakos et al. (2011) split holes into Type 1 (blown out, neither side seen, no measurable expansion), Type 2 (one side detected), and Type 3 (both sides, a closed ellipse). That sounds like a physical sequence, but it could be a detectability ladder: whether you see one cap, two, or none may depend mostly on resolution and on expansion speed relative to gas turbulence, not on three different kinds of object. The distinction matters, because training a classifier on Type labels might cause a systemic bias. So we

Table 1. Galaxy-held-out data splits. A cut is one standardized PV patch; a cut is “positive” if it carries any labeled shell pixel. Counts come from the pipeline’s split summary.

Split	Galaxies	Cuts	Positive	Negative
Train	10	15,540	11,023	4,517
Validation	4	2,664	1,780	884
Test	4	3,684	2,542	1,142
Stress (NGC 3031)	1	8,642	2,664	5,978
Total	19	30,530	18,009	12,521

ask directly whether the Types are physically distinct populations or a selection effect.

2. DATA

Everything starts from the THINGS survey (F. Walter et al. 2008), an H I 21 cm survey of nearby galaxies with the NRAO VLA. The cubes have angular resolution around 6 to 12 arcsec and $\sim 5 \text{ km s}^{-1}$ channels, fine enough to see the velocity split of an expanding shell in the closer galaxies. We read the celestial and spectral WCS straight from the FITS headers with Astropy (Astropy Collaboration 2022).

For labels we use Table 7 of I. Bagetakos et al. (2011), the published H I hole catalog. Each row gives a position, heliocentric velocity, diameter, expansion velocity (where measurable), position angle, axis ratio, and Type. Our loader, `load_bagetakos_table7`, parses the table, converts coordinates to degrees, cleans the axis ratios, and returns a tidy table per galaxy.

We work with 19 THINGS galaxies, a mix of large spirals (e.g. NGC 628, NGC 3184, NGC 6946) and dwarf irregulars (e.g. Holmberg I, Holmberg II, DDO 154). We hold out whole galaxies for evaluation so the model never sees a test galaxy during training. Ten galaxies go into training, four (Holmberg I, NGC 2366, NGC 4449, NGC 4736) into validation, four (DDO 53, Holmberg II, NGC 3184, NGC 7793) into testing, and NGC 3031 (M81) is set aside as a separate “stress” galaxy because it exhibits messy velocity signatures. After slicing and labeling (Section 3), this gives the four splits summarized in Table 1: 15,540 training cuts, 2,664 validation cuts, 3,684 test cuts, and 8,642 stress cuts, each split a disjoint set of galaxies.

3. THE CNN SHELL DETECTOR

This project has two graded techniques, and we give each its own section. The first, described here, is the detector: a convolutional neural network that slices PV cuts, labels them from the catalog, segments shell-like

structure, and post-processes its output into ranked sky candidates. The second, in Section 4, leaves the cubes behind and asks statistically whether the I. Bagetakos et al. (2011) Type labels are physical. Figure 1 sketches this detector end to end, from cube to candidate catalog.

3.1. Standardized PV cuts

A PV cut is defined by a center, a direction, a length, and a slit width. We place cuts on and around cataloged holes and also on a blind grid, then for each cut we sample the cube along the slit. Each column of the PV image is the mean spectrum in a thin 3 pixel slit, and each row is a velocity channel.

The hard part is that THINGS galaxies sit at very different distances, so the same physical hole spans a different number of pixels and channels in each cube. To make the cuts comparable, we standardize every cut to a fixed physical window: 5 kpc along the slit and 200 km s^{-1} in velocity, centered on the local moment-1 velocity along the cut. We convert the spatial window from kpc to arcsec with the small-angle relation $\theta \simeq L/D$, valid here since $L \ll D$,

$$\theta_{\text{arcsec}} = \frac{L_{\text{kpc}}}{D_{\text{Mpc}} 1000} \times 206265, \quad (1)$$

where the factor 1000 converts Mpc to kpc and 206,265 is the number of arcseconds in a radian; we then convert the result of Equation 1 to pixels using the cube pixel scale. We resample the velocity axis onto a fixed grid by linear interpolation, filling missing channels with the local finite median rather than noise. The result is a fixed 96×256 patch (velocity by position) for every cut, which is what the network sees. The per-bin resolution is about 2.08 km s^{-1} in velocity and 0.0195 kpc in position. Finally we z-score each patch using only its finite pixels, the standard per-feature standardization to zero mean and unit variance used to condition CNN inputs,

$$\tilde{x} = \frac{x - \mu_{\text{finite}}}{\sigma_{\text{finite}} + \epsilon}, \quad (2)$$

where ϵ guards against division by zero. We map any non-finite pixels to zero, so the input is roughly unit scale regardless of the cube’s brightness units.

3.2. Labels from the hole catalog

To turn a cataloged hole into a training mask, we project the slit samples into the hole’s elliptical frame. For a hole with semi-axes a and b and position angle, a sample at offset $(\Delta x, \Delta y)$ from the center has normalized radius given by the standard ellipse equation

$$\rho^2 = \left(\frac{u}{a}\right)^2 + \left(\frac{w}{b}\right)^2, \quad (3)$$

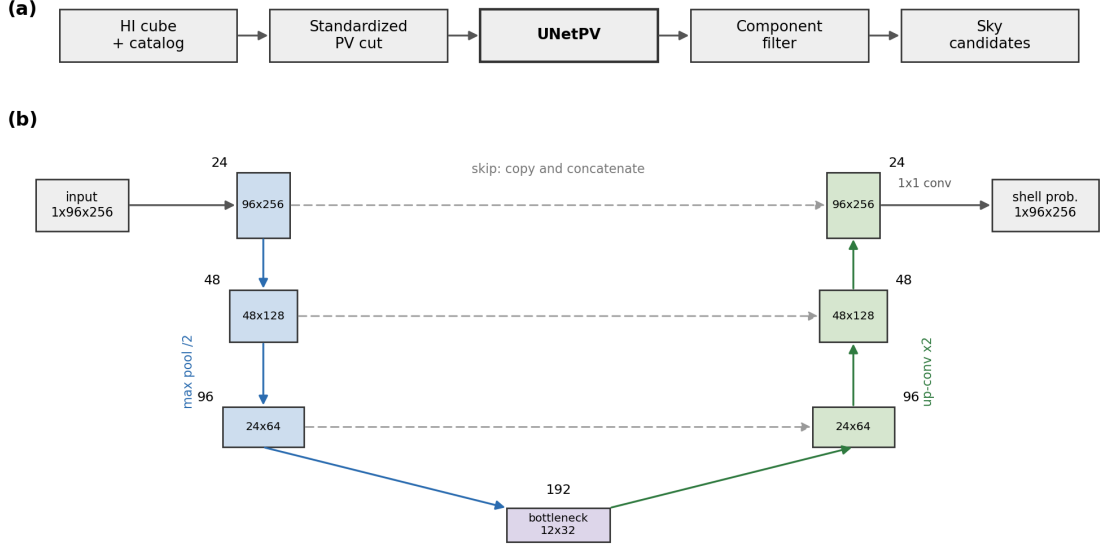


Figure 1. The detection pipeline (a) and the UNetPV architecture (b). THINGS HI cubes are sliced into standardized 96×256 PV cuts, labeled with beam-aware catalog masks, passed through the high-recall U-Net, and cleaned with connected-component filters before votes are projected onto the sky. Panel (b) shows the encoder-decoder with channel widths 24-48-96 and a 192-channel bottleneck; the per-stage block and training details are in Section 3.3 and `hishells_pv/models/unet.py`. The network is the sensitive detector, not the final catalog: physics filters and human review turn raw detections into credible candidates.

where u and w are the offsets along the major and minor axes, so $\rho = 1$ is the rim. Columns with $\rho^2 \leq 1$ (Equation 3) are inside the hole on that cut. The velocity band we paint depends on the Type, and this is where the I. Bagetakos et al. (2011) physics enters directly. For Type 1 holes (cavities with no measured expansion) we paint a thin fixed band of about 10 km s^{-1} around the hole velocity. For Type 2 and Type 3 holes (expanding) we paint a half-ellipse in the PV plane that tracks the projected expansion velocity. For a shell expanding uniformly at V_{exp} , the line-of-sight velocity at projected fractional radius ρ is $V_{\text{exp}} \cos \theta = V_{\text{exp}} \sqrt{1 - \rho^2}$, the classic expanding-shell ellipse in a PV cut (I. Bagetakos et al. 2011), so we paint a band of half-width

$$v_{1/2}(\rho) = (V_{\text{exp}} + v_{\text{pad}}) \sqrt{1 - \rho^2}, \quad (4)$$

which is widest at the hole center and closes off at the rim ($\rho = 1$). After painting, we run a beam-aware cleanup that erases interior label fragments narrower than the beam, since the model should not be asked to find structure the data cannot resolve.

We also generate negatives on purpose: cuts offset from holes, cuts at wrong angles, cuts at wrong velocities, grazing cuts, and random background cuts in gas-free regions, so the model does not learn to call everything a shell.

3.3. The U-Net detector

We treat shell finding as image segmentation: given a standardized PV patch (Equation 2), output a per-pixel probability that the pixel belongs to a shell. The model is a U-Net (O. Ronneberger et al. 2015), which is the standard architecture for this kind of dense prediction.

Briefly: a convolutional neural network slides small learnable filters over an image, so each layer responds to local patterns (edges, arcs, rings) wherever they sit. A U-Net has two halves. The encoder applies convolutions and downsamples with max pooling to build a coarse, abstract description of the patch; the decoder upsamples back to full resolution, with skip connections that paste the matching high-resolution encoder features back in at each step, so the output mask is both semantically aware and sharp. Our implementation, UNetPV, uses 24 base filters, depth 3 (channel widths 24, 48, 96, with a 192-channel bottleneck), two 3×3 convolution blocks per stage each followed by batch normalization and a ReLU nonlinearity, 0.1 dropout for regularization, transposed convolutions for upsampling, and a final 1×1 convolution that emits one logit per pixel (panel (b) of Figure 1 draws this architecture, matching `hishells_pv/models/unet.py`). We apply the sigmoid afterward to get probabilities.

The loss is the key design choice, because we care far more about not missing shells (recall) than about perfectly clean masks. We use a weighted sum of binary cross-entropy and a soft Tversky loss (S. S. M. Salehi et al. 2017). The Tversky score is

$$T = \frac{TP + s}{TP + \alpha FP + \beta FN + s}, \quad (5)$$

where TP, FP, and FN are soft (probability-weighted) true positives, false positives, and false negatives, s is a smoothing constant, and the Tversky loss is $1 - T$. The index reduces to the Dice/F1 overlap when $\alpha = \beta = 0.5$; following S. S. M. Salehi et al. (2017) we instead set $\alpha = 0.3$ and $\beta = 0.7$, so false negatives (missed shell pixels) cost more than false positives. The full loss is the standard compound of a pixel-wise term and a region-overlap term,

$$\mathcal{L} = 0.5 \mathcal{L}_{\text{BCE}} + 0.5 (1 - T), \quad (6)$$

which gives a recall-leaning but still calibrated objective. We train with AdamW (I. Loshchilov & F. Hutter 2019) at a learning rate of 10^{-3} and weight decay 10^{-4} , batch size 8, a fixed random seed of 42, and no data augmentation (the cut generation already injects plenty of geometric variety). The run reported here is a twenty-epoch demonstration run on CPU using PyTorch (A. Paszke et al. 2019).

3.4. From probabilities to a catalog

Three steps turn raw probability maps into something a person can follow up.

Calibration. We sweep a grid of thresholds from 0.02 to 0.6 on the validation set and, at each threshold, score detection quality at the patch level (a patch counts as a hit if any shell pixel is recovered) using precision, recall, and their harmonic mean,

$$F_1 = \frac{2PR}{P+R}. \quad (7)$$

We pick the threshold that maximizes patch F_1 (Equation 7) subject to a recall floor.

Components. On a thresholded map we run connected-component labeling, measure each blob’s area and total probability mass, drop blobs below a minimum area, and rank what remains by probability mass.

Sky aggregation. A single hole is crossed by many cuts, so we project the per-cut evidence back onto the image plane: for each cut we take the max probability down each column and “splat” that weight onto the sky at the column’s pixel. We accumulate all cuts into one vote map, smooth it with a Gaussian ($\sigma = 5$ pixels), find

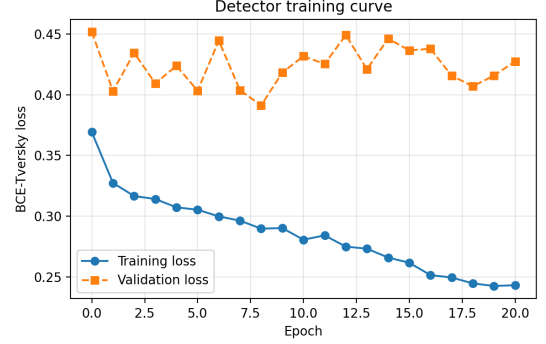


Figure 2. Training and validation BCE-Tversky loss per epoch for the twenty-epoch run. Training loss (solid) falls steadily; validation loss (dashed) is roughly flat, which says the model is still in the underfit regime and would benefit from longer training.

peaks with non-maximum suppression (radius 12 pixels, floor at the 90th percentile), and drop peaks smaller than 2.5 beams or too thin to be a shell. The output is a ranked sky catalog per galaxy.

3.5. What we did not use

We considered classical parametric shell searches and Hough/template matching (S. Mashchenko & N. St-Louis 2002; S. Ehlerová & J. Palouš 2005), but those need a guessed shell shape up front and struggle with the partial or blended shells that dominate the catalog. We also rejected a box-detector network, since bounding boxes throw away the arc shape that makes a shell a shell, whereas segmentation keeps it. We kept the model small and recall-leaning rather than chasing a clean catalog in one shot: a first-pass finder feeding a human is more honest about what short PV cuts can support.

3.6. Detection performance

The model trains cleanly. Figure 2 shows the BCE-Tversky loss (Equations 5 and 6) dropping from 0.43 to 0.29 on the training set over twenty epochs while the validation loss sits flat around 0.40 to 0.43: the model is learning, not yet overfitting, with room to improve over more epochs. Unless stated otherwise, every detector number in this section comes from our primary PyTorch run (`pv_unet_real`); the one exception is the patch confusion matrix below, which we read from a parallel TensorFlow run as an independent cross-check.

What matters for a finder is detection recall, and that is strong from the start. Figure 3 shows patch-level precision, recall, and F1 on the validation set at the fixed training-monitor threshold (0.075): recall sits near 1.0, F1 near 0.93, and precision near 0.88 across all twenty epochs, so the network almost never misses a labeled shell. These are per-epoch training-monitor

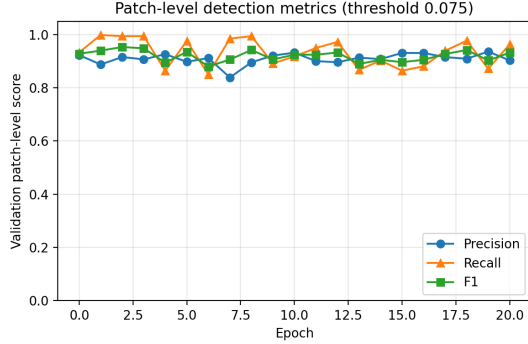


Figure 3. Validation patch-level precision, recall, and F1 at threshold 0.075 over training. Recall stays near unity and F1 near 0.93, confirming the detector is high-recall by construction.

scores rather than the calibrated patch metrics reported below; the monitor and the calibration sweep do not score patches identically, so the higher precision and F1 here are not directly comparable to the calibrated operating point. The pixel-level metrics are lower (precision ~ 0.2 to 0.3 , recall ~ 0.4 to 0.5), as expected: locating a shell is far easier than painting its exact mask on a short, noisy cut, which we did not optimize for.

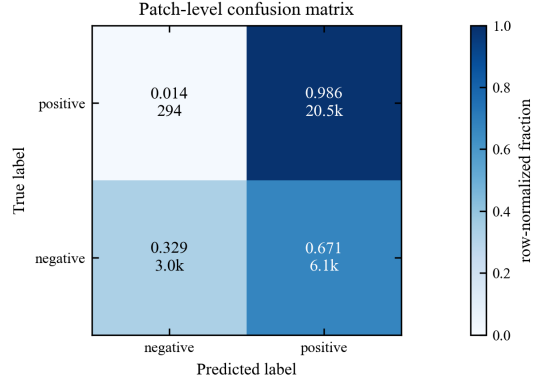
The patch-level confusion matrix (Figure 4) makes the tradeoff explicit: almost every positive patch is recovered (recall 0.986), while about two thirds of negative patches leak through as false positives, the deliberate cost of a high-recall first pass.

Figure 5 makes this concrete on held-out galaxies, comparing the prediction against the catalog label painted by the rules of Section 3.2 (Equations 3 and 4): the model consistently lands on the right structure even when the exact mask is ragged.

The 0.075 threshold above is only what we watch during training; for deployment we calibrate it properly (Section 3.4). Calibrating on the validation set (2,664 cuts) picks an operating point of 0.40, where patch precision, recall, and F_1 (Equation 7) are 0.80, 0.97, and 0.88. Table 2 walks the full sweep: pushing down toward 0.02 raises patch recall above 0.99 at the cost of precision, while patch F_1 is flat to within about one percent from 0.3 to 0.6, so the detector is not on a knife edge.

3.7. Sky-plane candidate catalogs

Aggregating the per-cut votes back onto the sky produces a candidate map per galaxy. In each map (Figure 6) the vote density traces the gas disk and the ranked detections (cyan circles) land on the structured, hole-rich inner disk, where I. Bagetakos et al. (2011) find most of their holes. Across the held-out galaxies the detector returns the candidate counts in Table 3: 127 in NGC



threshold=0.075; recall=0.986; precision=0.769; specificity=0.329

Figure 4. Patch-level confusion matrix for the high-recall detector at threshold 0.075, pooled over the validation, test, and stress galaxies (rows are true labels, columns predicted). Recall is 0.986 and precision 0.769 (TP = 20520, FN = 294, TN = 3014, FP = 6148). These counts come from the parallel TensorFlow run used as the cross-check noted at the start of this section; that run regenerates its own PV cuts, so the totals are not one-to-one with the cut counts in Table 1. Our primary PyTorch detector shows the same high-recall, lower-precision pattern by design.

Table 2. Validation detection metrics versus probability threshold, from the calibration sweep. “Patch” scores a cut as a hit if any shell pixel is recovered; “pixel” scores the segmentation mask. The chosen operating point is $t = 0.40$.

t	Patch			Pixel		
	P	R	F_1	P	R	F_1
0.02	0.75	0.99	0.86	0.24	0.53	0.33
0.10	0.78	0.98	0.87	0.28	0.47	0.35
0.20	0.79	0.98	0.87	0.29	0.45	0.35
0.30	0.80	0.97	0.88	0.30	0.43	0.36
0.40	0.80	0.97	0.88	0.31	0.42	0.36
0.50	0.80	0.97	0.88	0.32	0.41	0.36
0.60	0.81	0.96	0.88	0.33	0.40	0.36

7793, 123 in NGC 3184, 77 in Holmberg II, 8 in the small dwarf DDO 53, and 436 in the NGC 3031 stress galaxy. In total the test split yields 8,627 raw candidate components across 3,684 cuts before sky-plane merging.

The candidate vote scores (Table 3) behave sensibly: gas-rich spirals and the busy stress galaxy reach the highest scores (max 6.4, 8.5, 8.9), while the gas-poor dwarf DDO 53 yields only low-score peaks (max 1.9).

Table 3. Ranked sky-plane shell candidates per galaxy after vote aggregation, non-maximum suppression, and the beam-size and thickness cuts. “Candidates” counts the accepted peaks in each galaxy’s vote map; the vote-score columns give the integrated probability mass at those peaks, read from the run’s `detections_*.json` files. Gas-rich spirals and the busy stress galaxy reach much higher scores than the gas-poor dwarf DDO 53.

Galaxy	Morphology	Split	Candidates	Vote score		
				Min	Median	Max
NGC 7793	Spiral	Test	127	0.04	0.21	4.31
NGC 3184	Spiral	Test	123	0.02	0.18	8.51
Holmberg II	Dwarf Irr	Test	77	0.06	0.73	6.39
DDO 53	Dwarf Irr	Test	8	0.03	0.12	1.88
NGC 3031 (M81)	Spiral	Stress	436	0.02	0.12	8.88

Candidate radii cluster at 10 to 20 pixels with no pathological trend of score with size, as intended from the beam-size and thickness filters.

3.8. Detector uncertainties

The detection metrics are computed on whole held-out galaxies, not random cuts, so they reflect generalization rather than memorization, and the calibration grid (Section 3.4) doubles as a sensitivity analysis. The biggest caveat is the short twenty-epoch CPU run, which is why pixel-level precision is low; the patch-level recall we actually use is already saturated.

4. ARE THE BAGETAKOS TYPES PHYSICAL?

For this second technique we leave the cubes behind and work only with the I. Bagetakos et al. (2011) catalog, asking whether the Type 1/2/3 split marks three real populations or just a detectability cut. We attack the question from three angles, and the answer leans clearly toward “mostly detectability.”

4.1. Method: three independent tests

First, unsupervised clustering. We standardize the physical hole features that do not directly encode expansion, fit Gaussian mixture models for $K = 1$ to 8 components, and select K with the Bayesian Information Criterion (G. Schwarz 1978),

$$\text{BIC} = k \ln n - 2 \ln \hat{L}, \quad (8)$$

where k is the number of parameters, n the sample size, and $\hat{L}$ the maximized likelihood; the $k \ln n$ term penalizes added components, so lower BIC is preferred. We then compare the data-driven clusters to the published Types with the adjusted Rand index (ARI) and normalized mutual information. If the Types were natu-

ral groupings, the unsupervised clusters should recover them.

Second, regression. We build a simple detectability proxy, the hole diameter in beams, $\log_{10}(d_{\text{pc}}/\text{Res}_{\text{pc}})$, and fit nested logistic regressions for the probability that a hole is labeled Type 1. We start with the proxy alone, then add galaxy fixed effects and physical controls. If Type is physical, the proxy’s coefficient should shrink once we control for the real physics. If it is a detectability cut, the coefficient should stay large.

Third, supervised learning. We train a random forest to predict Type from the full feature set, using grouped cross-validation (GroupKFold) so all holes from one galaxy stay in the same fold. We compare its accuracy to two baselines: a model that sees only the detectability proxy, and a dumb majority-class model. We also back all of this with nonparametric tests (Anderson-Darling, Kruskal-Wallis, Mann-Whitney with Cliff’s delta) and Benjamini-Hochberg false-discovery-rate control, because the feature distributions are not Gaussian.

4.2. Results: the Types track detectability

Following the three tests set out in Section 4.1, we first compare the physical hole features across Types with nonparametric tests (Table 4). The Kruskal-Wallis omnibus rejects a common distribution for nearly every feature, but that signal is dominated by Type 1, whose V_{exp} -derived quantities are assumed rather than measured. The sharper test is Type 2 versus Type 3, the two genuinely measured Types: only hole size separates them ($\log d_{\text{pc}}$ and $\log M_{\text{HI}}$ reach $q < 0.05$ with Cliff’s $|\delta| \geq 0.15$), and size is exactly what sets detectability, while shape and density (R , axis ratio, $\log n_{\text{HI}}$) show no meaningful Type 2 vs Type 3 difference.

Table 4. Per-feature Type comparisons from the catalog analysis. KW q is the Benjamini-Hochberg-corrected Kruskal-Wallis omnibus q -value; q_{ij} and δ_{ij} are corrected Mann-Whitney U q -values and Cliff’s delta effect sizes for Type i vs j . Daggers mark features derived from V_{exp} (assumed for Type 1); the double dagger marks the detectability proxy (cavity diameter in beams). For the measured Types (2 vs 3), only $\log d_{\text{pc}}$ and $\log M_{\text{HI}}$ pass $q < 0.05$ with $|\delta| \geq 0.15$.

Feature	KW q	q_{12}	q_{13}	q_{23}	δ_{12}	δ_{13}	δ_{23}
$\log d_{\text{pc}}$	7×10^{-39}	6×10^{-19}	3×10^{-31}	1.5×10^{-4}	+0.39	+0.60	+0.23
axis ratio	8×10^{-4}	0.22	1.5×10^{-4}	0.017	−0.05	−0.20	−0.14
R	0.81	0.70	0.68	0.64	−0.02	+0.02	+0.03
$\log n_{\text{HI}}$	5×10^{-11}	4×10^{-7}	1×10^{-8}	0.15	−0.22	−0.30	−0.09
$\log t_{\text{kin}}^\dagger$	1×10^{-73}	2×10^{-48}	1×10^{-45}	0.038	+0.63	+0.73	+0.12
$\log E^\dagger$	1×10^{-9}	0.004	2×10^{-10}	0.003	+0.13	+0.33	+0.18
$\log M_{\text{HI}}$	2×10^{-20}	4×10^{-10}	2×10^{-17}	0.008	+0.27	+0.44	+0.16
$d/\text{beam}^\ddagger$	3×10^{-34}	1×10^{-18}	7×10^{-26}	0.002	+0.38	+0.55	+0.19

The unsupervised clustering does not recover the Types. Figure 7 shows the BIC (Equation 8) preferring $K = 5$ Gaussian components, not the 3 published Types, and the clusters that the data do want barely overlap with the Type labels (adjusted Rand index near 0.02, i.e. essentially chance). If the three Types were natural groups in feature space, clustering should have recovered them; it did not.

The regression points the same way. Figure 8 shows the probability of a Type 1 label rising steeply with how well a hole is resolved (its diameter in beams). The pooled logistic coefficient on the detectability proxy is large and positive, $\beta \approx +2.95$. The telling part is what happens when we add galaxy fixed effects and physical controls: instead of shrinking (as expected if Type were physical and the proxy a nuisance), the coefficient grows to roughly +5.6. Resolution is not a confounder here, it is one of the main things the label responds to.

The supervised random forest seals it. Figure 9 shows the confusion matrix: the model recovers Type 1 reasonably (0.79 recall) but collapses on Type 2 (0.07) and is barely better than a coin flip on Type 3 (0.51), constantly confusing the two expanding Types. On grouped cross-validation the full-feature forest reaches accuracy 0.568, only a hair above a detectability-only model at 0.531, and it actually loses to a dumb “always predict Type 1” baseline at 0.598. A real three-population structure should be learnable well above the majority baseline. This one is not.

Finally, the physics is consistent with detectability. Figure 10 shows the measured expansion velocities of the resolved-rim Type 2 and Type 3 holes piling up right at the turbulent floor of the gas. We adopt a floor of $\sigma_{\text{gas}} \approx 7$ to 10 km s^{-1} , the typical H I velocity dispersion in nearby disks (A. K. Leroy et al. 2008; D. Tam-

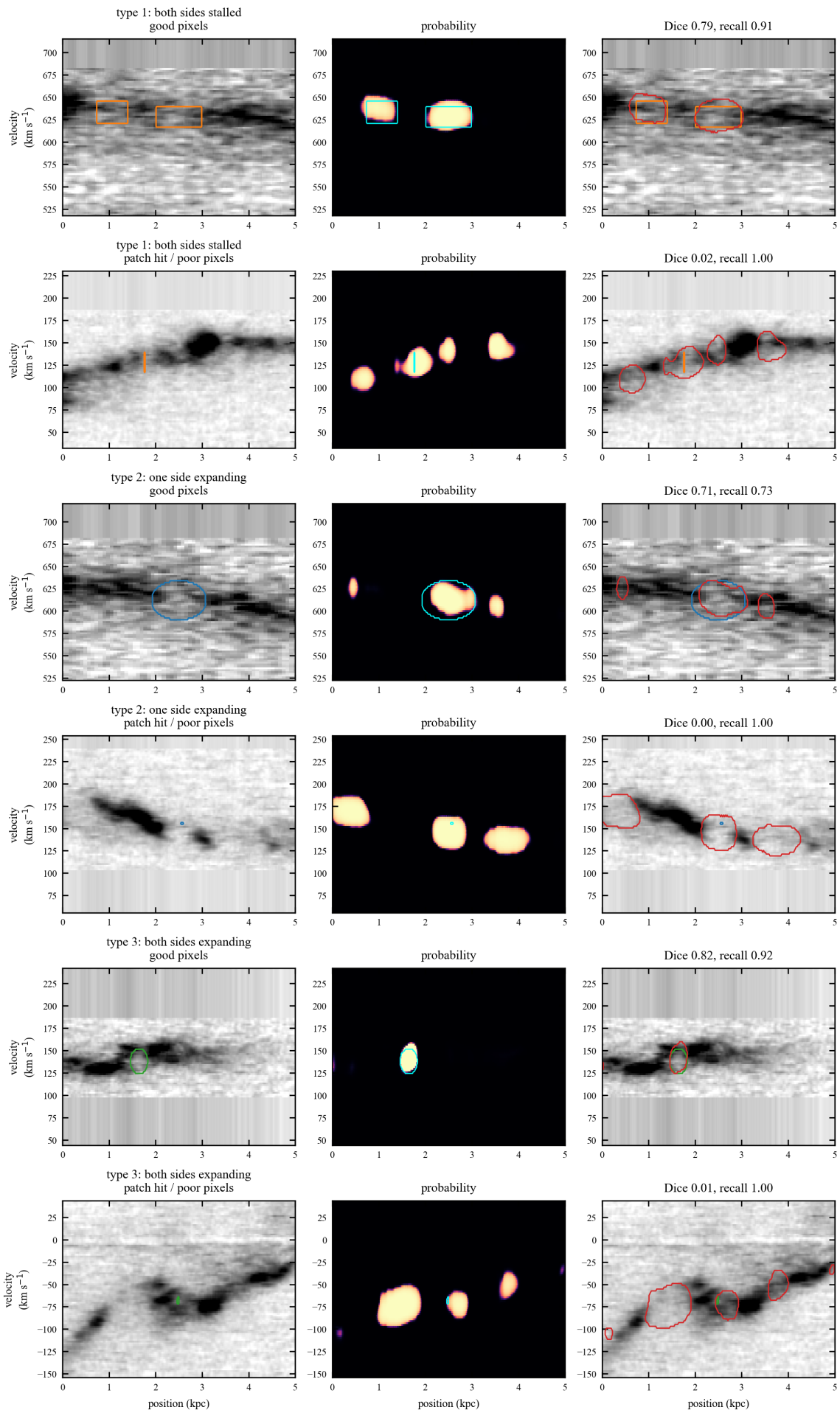
burro et al. 2009), which is the same turbulence that stellar feedback is thought to sustain (C. Bacchini et al. 2020). A large fraction sit inside that band (38.8% of Type 2 and 55.3% of Type 3), meaning their “expansion” is barely distinguishable from random gas motion. Whether a hole gets a one-sided (Type 2) or two-sided (Type 3) label near that floor is exactly the kind of coin flip you would expect from noise and resolution, not from a clean physical boundary.

Tying it together, the Type fractions per galaxy track the THINGS physical beam size (the resolution) more than anything intrinsic: ordered by Res_{pc} , the Type 1/2/3 mix shifts steadily with resolution. Galaxies are not made of different kinds of holes, they are observed at different resolutions. I. Bagetakos et al. (2011) themselves note that the count of Type 3 holes is strongly affected by resolution and that Type 1 holes have likely stalled at the gas velocity dispersion; our analysis takes that caveat to its conclusion and quantifies it.

All four tests (the per-feature comparison in Table 4, the clustering in Figure 7, the regression in Figure 8, and the random forest in Figure 9) are independent and point the same way, so together they say the Type label is mostly a detectability cut rather than a clean physical split.

4.3. Catalog-analysis uncertainties

The random-forest accuracies come from grouped cross-validation (so the majority-baseline comparison is apples to apples), the hypothesis tests use Benjamini-Hochberg FDR control rather than raw p-values, and the agreement metrics (ARI, NMI) are chance-corrected.



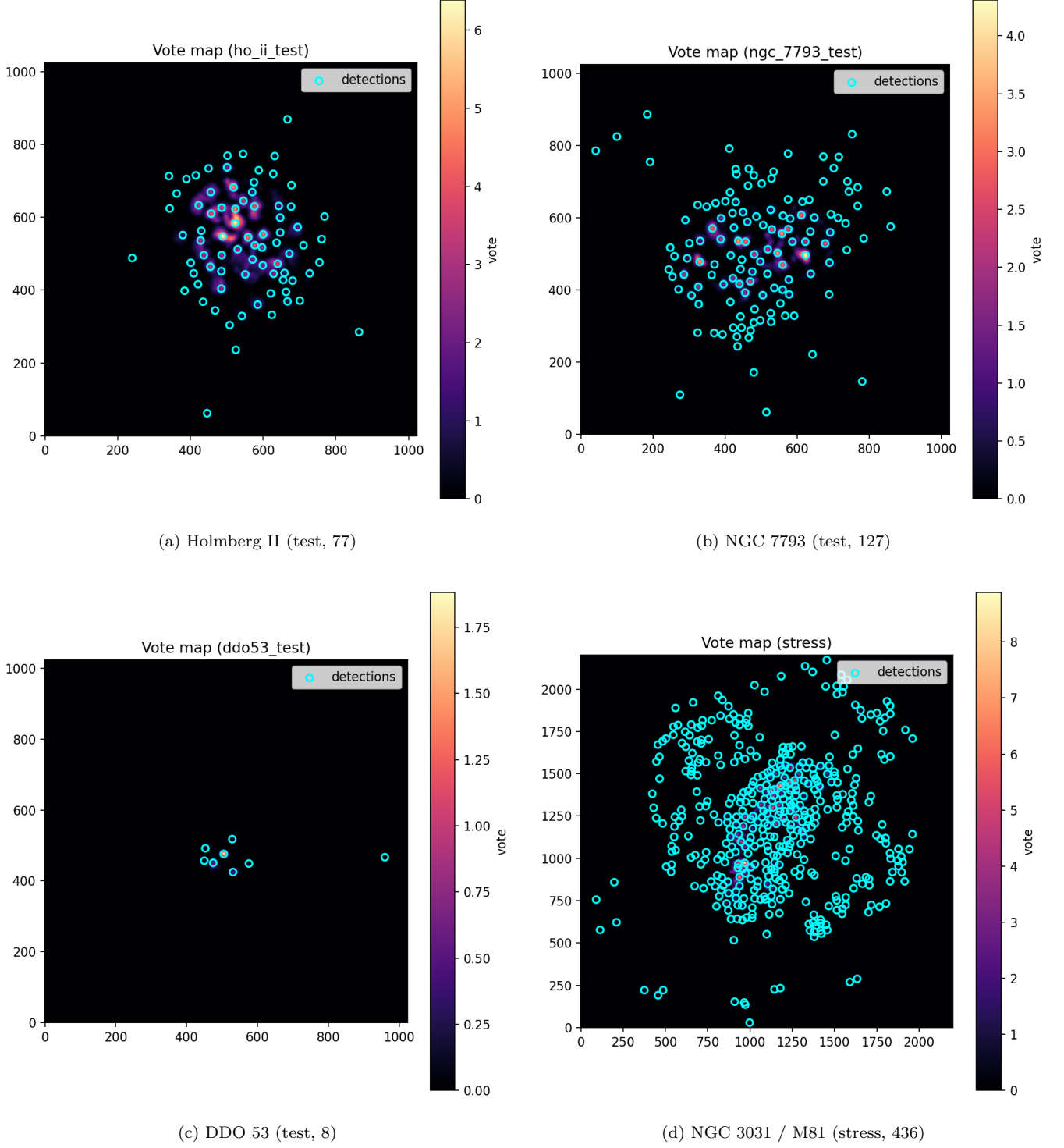


Figure 6. Sky-plane vote maps for four of the five held-out and stress galaxies in Table 3 (NGC 3184 not shown), with accepted candidate counts in parentheses. Color is the smoothed detection vote; the candidate density follows the H I disk and scales with gas richness, from the sparse dwarf DDO 53 to the busy NGC 3031.

5. NEXT STEPS

The obvious first move is to train longer on a GPU; twenty CPU epochs are a proof of concept and the flat

validation loss says there is headroom. The most interesting direction is uncertainty: we built dropout into the network, so Monte Carlo dropout (Y. Gal & Z. Ghahra-

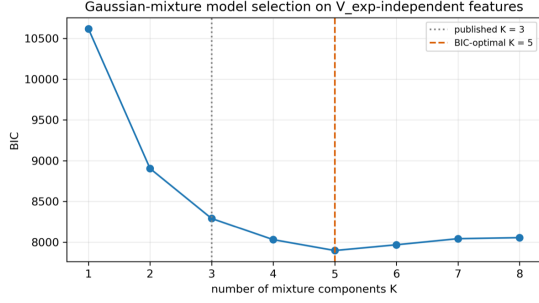


Figure 7. Gaussian-mixture BIC versus number of components K on the expansion-independent hole features. BIC prefers $K = 5$ (dashed) over the 3 published Bagetakos Types (dotted), and the resulting clusters do not match the Types (ARI ≈ 0.02).

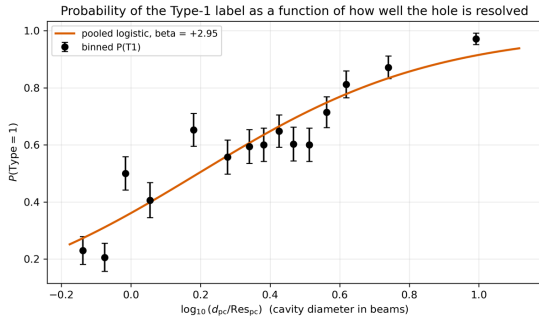


Figure 8. Probability of the Type 1 label versus how well the hole is resolved, $\log_{10}(d_{\text{pc}}/\text{Res}_{\text{pc}})$ (cavity diameter in beams). Points are binned empirical $P(\text{Type} = 1)$ with Poisson error bars; the curve is the pooled logistic fit ($\beta = +2.95$). Better-resolved holes are far more likely to be labeled Type 1.

mani 2016; Y. Gal 2016a) would make it an approximate Bayesian model and attach a calibrated confidence to each candidate (A. Kendall & Y. Gal 2017). A natural extension is a 3D CNN on the full cube (L. Wang et al. 2025), seeing the shell in space and velocity at once rather than one slit at a time. We would also push beyond THINGS to WALLABY and MeerKAT, cross-match candidates against H II regions and other star-formation tracers, and fix an approximate labeling assumption (we treat the catalog diameter as a full diameter). The single biggest improvement would be a small set of human-vetted candidates to fine-tune on, since the model is only as good as its labels, and those labels carry selection effects.

6. SOFTWARE AND REPRODUCIBILITY

All code is in the `hishells_pv` package, public at <https://github.com/Ericturip/HIShell>. The package is organized by stage: `catalog/` loads the I. Bagetakos et al. (2011) table (`load_bagetakos.table7`);

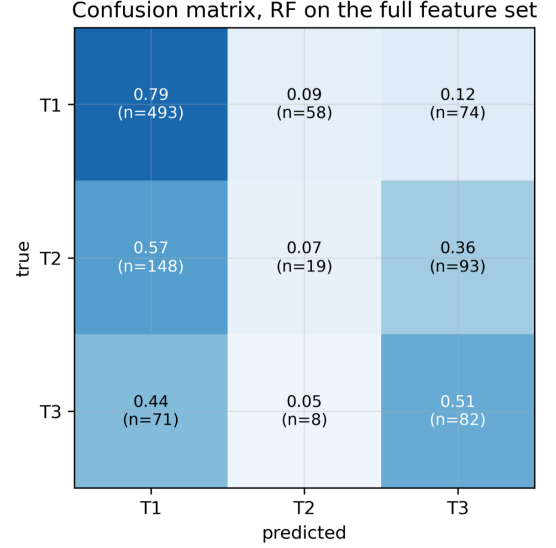


Figure 9. Random-forest confusion matrix (grouped cross-validation, full feature set). Rows are true Types, cells show recall and counts. Type 1 is recovered, but Types 2 and 3 are routinely confused with each other and with Type 1, which is what you expect if the expanding Types are one population split by detectability.

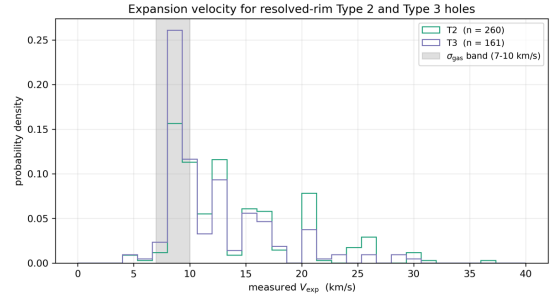


Figure 10. Expansion velocity distributions for resolved-rim Type 2 ($n = 260$) and Type 3 ($n = 161$) holes. The shaded band is our adopted gas turbulent floor $\sigma_{\text{gas}} \approx 7$ to 10 km s^{-1} , the typical H I dispersion in nearby disks (A. K. Leroy et al. 2008; D. Tamburro et al. 2009). Both distributions pile up at the floor, where the measured expansion is hard to tell from turbulence.

`pv/` makes and standardizes cuts and paints labels; `data/dataset.py` builds the PyTorch dataset and z-score normalization; `models/` holds the network (UNetPV), the loss (BCETverskyLoss), and the metrics; `train/` runs training; and `infer/` does inference, threshold calibration, component extraction, and sky aggregation. Alongside the PyTorch detector reported here, the repo ships a TensorFlow training path and command-line drivers (`generate`, `train`, `infer`, `calibrate`, `postprocess`, `aggregate-all`); the selection-effects study lives in

`notebooks/shell_type_selection_effects.ipynb`.

Detector-metric figures come from the run’s `history_torch.csv`, the vote maps from the `aggregate_*` outputs, and the catalog figures from the notebook. A `pytest` suite covers catalog parsing, the U-Net forward pass, the losses, the dataset, and a synthetic end-to-end run, so the pipeline can be checked without proprietary data. Given the THINGS cubes, the Bagetakos catalog, and this code, every result above can be reproduced.

ACKNOWLEDGMENTS

This work uses data from the THINGS survey (F. Walter et al. 2008) and the H I hole catalog of I. Bagetakos et al. (2011). It makes use of NumPy (C. R. Harris et al. 2020), SciPy (P. Virtanen et al. 2020), Astropy (Astropy Collaboration 2022), scikit-learn (F. Pedregosa et al. 2011), statsmodels (S. Seabold & J. Perktold 2010), and PyTorch (A. Paszke et al. 2019).

APPENDIX

A. GENERATIVE AI USE

In keeping with the course policy, we disclose our use of generative AI. We used Codex Desktop (OpenAI, gpt-5.5) as coding and editing assistants. Common prompts were narrow and practical: refactor a pipeline stage into the `hishells_pv` package, add `pytest` coverage for the U-Net and losses, debug the vote-map aggregation and tidy the analysis notebook. We designed the study, chose every method, ran the full pipeline, and produced and checked all results ourselves: each AI-touched code path is covered by tests and was run end to end, and every number here was read from the pipeline’s own output files (for example, `history_torch.csv` and the per-galaxy `detections_*.json`) rather than generated by a model. Full prompt-and-response transcripts are in the repository at https://github.com/Ericturnip/HIShell/blob/main/AI_TRANSCRIPT_APPENDIX.md. We remain fully responsible for all content.

REFERENCES

- Astropy Collaboration. 2022, *ApJ*, 935, 167, doi: [10.3847/1538-4357/ac7c74](https://doi.org/10.3847/1538-4357/ac7c74)
- Bacchini, C., Fraternali, F., Iorio, G., et al. 2020, *A&A*, 641, A70, doi: [10.1051/0004-6361/202038223](https://doi.org/10.1051/0004-6361/202038223)
- Bagetakos, I., Brinks, E., Walter, F., et al. 2011, *AJ*, 141, 23, doi: [10.1088/0004-6256/141/1/23](https://doi.org/10.1088/0004-6256/141/1/23)
- Boomsma, R., Oosterloo, T. A., Fraternali, F., van der Hulst, J. M., & Sancisi, R. 2008, *A&A*, 490, 555, doi: [10.1051/0004-6361:200810120](https://doi.org/10.1051/0004-6361:200810120)
- Brinks, E., & Bajaja, E. 1986, *A&A*, 169, 14, <https://ui.adsabs.harvard.edu/abs/1986A&A...169...14B>
- Ehlerová, S., & Palouš, J. 2005, *A&A*, 437, 101, doi: [10.1051/0004-6361:20034389](https://doi.org/10.1051/0004-6361:20034389)
- Gal, Y. 2016a, PhD thesis, University of Cambridge, <https://www.cs.ox.ac.uk/people/yarin.gal/website/thesis/thesis.pdf>
- Gal, Y., & Ghahramani, Z. 2016, in *Proceedings of Machine Learning Research*, Vol. 48, *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 1050–1059. <https://arxiv.org/abs/1506.02142>
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. 2020, *Nature*, 585, 357, doi: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2)
- Kendall, A., & Gal, Y. 2017, in *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30, 5574–5584. <https://arxiv.org/abs/1703.04977>
- Leroy, A. K., Walter, F., Brinks, E., et al. 2008, *AJ*, 136, 2782, doi: [10.1088/0004-6256/136/6/2782](https://doi.org/10.1088/0004-6256/136/6/2782)
- Loshchilov, I., & Hutter, F. 2019, in *International Conference on Learning Representations (ICLR)*, <https://arxiv.org/abs/1711.05101>
- Mashchenko, S., & St-Louis, N. 2002, in *Astronomical Society of the Pacific Conference Series*, Vol. 260, *Interacting Winds from Massive Stars*, ed. A. F. J. Moffat & N. St-Louis, 65. <https://ui.adsabs.harvard.edu/abs/2002ASPC..260..65M>
- Paszke, A., Gross, S., Massa, F., et al. 2019, in *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32, 8024–8035. <https://arxiv.org/abs/1912.01703>
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. 2011, *Journal of Machine Learning Research*, 12, 2825. <https://arxiv.org/abs/1201.0490>
- Ronneberger, O., Fischer, P., & Brox, T. 2015, in *Lecture Notes in Computer Science*, Vol. 9351, *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 234–241, doi: [10.1007/978-3-319-24574-4_28](https://doi.org/10.1007/978-3-319-24574-4_28)
- Salehi, S. S. M., Erdogmus, D., & Gholipour, A. 2017, in *Machine Learning in Medical Imaging (MLMI)*, 379–387, doi: [10.1007/978-3-319-67389-9_44](https://doi.org/10.1007/978-3-319-67389-9_44)
- Schwarz, G. 1978, *Annals of Statistics*, 6, 461, doi: [10.1214/aos/1176344136](https://doi.org/10.1214/aos/1176344136)

- Seabold, S., & Perktold, J. 2010, in Proceedings of the 9th Python in Science Conference (SciPy), 92–96, doi: [10.25080/Majora-92bf1922-011](https://doi.org/10.25080/Majora-92bf1922-011)
- Tamburro, D., Rix, H.-W., Leroy, A. K., et al. 2009, AJ, 137, 4424, doi: [10.1088/0004-6256/137/5/4424](https://doi.org/10.1088/0004-6256/137/5/4424)
- Van Oort, C. M., Xu, D., Offner, S. S. R., & Gutermuth, R. A. 2019, ApJ, 880, 83, doi: [10.3847/1538-4357/ab275e](https://doi.org/10.3847/1538-4357/ab275e)
- Virtanen, P., Gommers, R., Oliphant, T. E., et al. 2020, Nature Methods, 17, 261, doi: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2)
- Walter, F., Brinks, E., de Blok, W. J. G., et al. 2008, AJ, 136, 2563, doi: [10.1088/0004-6256/136/6/2563](https://doi.org/10.1088/0004-6256/136/6/2563)
- Wang, L., Wong, O. I., Westmeier, T., et al. 2025, PASA, 42, e033, doi: [10.1017/pasa.2025.14](https://doi.org/10.1017/pasa.2025.14)